

VL-326: Fundamentals of Processor Architecture and SoC Design

Academic Year 2025–26 | MVLSI | Instructor: Dr. Bikram Paul

Duration: 1 month; **HDL:** Verilog / SystemVerilog; **Target:** Xilinx FPGA.

1 Common constraints for all projects:

- The **5-stage RV32I pipeline** (IF, ID, EX, MEM, WB) must be retained and must pass all baseline simulation tests before any modification.
- RTL must be written in synthesizable Verilog or SystemVerilog. Behavioural-only constructs that do not synthesize are not accepted.
- Synthesis and implementation must target the unavailable FPGA boards in lab (TAs will help you in this regards). Reports must include LUT count, FF count, BRAM usage, and $F_{\max}$.
- Every project must state a specific *research question* and provide a quantitative answer in the writeup.
- Comparative analysis must be produced among base RV32I and your project problem.

2 Individual Projects (P1–P21)

2.1 Category A — CPU Micro-Architecture (P1–P7)

P1: Advanced branch predictor comparison

Implement three direction predictors — bimodal (2-bit saturating counter), local history (2-level), and gshare — sharing a common Branch Target Buffer (BTB). Synthesize all three configurations, measure misprediction rate and $F_{\max}$, and plot a CPI vs. predictor-area trade-off curve. A hardware MUX should allow hot-swapping predictor configurations at reset.

***Research question:** Does local history or global history dominate for integer RV32I workloads at equal hardware cost?*

P2: Compiler-directed static branch prediction

Modify the pipeline to implement hint-bit static prediction using RISC-V's opcode-bit encoding. Benchmark against the default predict-not-taken policy using five hand-written RISC-V programs (fibonacci, selection sort, strlen, matrix multiply, bitcount). Report misprediction rate and code-size overhead for each program.

***Research question:** How much IPC improvement is achievable without dynamic hardware, and at what code-size cost?*

P3: Scoreboard-based dynamic scheduling

Implement a scoreboard unit (Thornton 1964 style) over the base pipeline for in-order issue but out-of-order completion. Implement WAR and WAW detection and resolution. Report the critical-path change vs. the base core and the fraction of stalls converted to other hazard classes.

Research question: What fraction of stalls in a standard RV32I workload are truly eliminated versus merely converted?

P4: Pipeline with branch delay slot emulation

Add a delay-slot buffer behind the branch execute stage so the instruction immediately after a branch always completes (MIPS-style). Implement a software-visible CSR bit to enable/disable the slot. Write an assembler post-pass to legally fill delay slots and measure the fill rate.

Research question: What is the net CPI improvement under real instruction schedules and what fraction of delay slots can be legally filled?

P5: Register renaming with a 48-entry physical register file

Extend the pipeline with a rename table mapping 32 architectural registers to 48 physical registers. Implement allocation on decode and free-on-commit. Restrict to RAW elimination only (no OoO issue). Compare stall-cycle reduction to the baseline forwarding unit.

Research question: How much does renaming alone reduce stall cycles in a hazard-dense loop, without out-of-order issue?

P6: Superscalar 2-wide fetch and decode

Widen fetch and decode to issue two instructions per cycle while keeping execute and memory-access single-issue. Synthesize 1-wide and 2-wide variants and report LUT and $F_{\max}$ changes. Characterise the effective IPC under realistic instruction mixes.

Research question: What is the achievable IPC under 2-wide fetch when execute remains 1-wide — where does the bottleneck move?

P7: M-mode exception and interrupt handling

Implement the RISC-V M-mode privileged subset: `mstatus`, `mepc`, `mcause`, `mtvec` CSRs, plus `ecall`, `ebreak`, and `mret` instructions, and a single external interrupt line with GPIO stimulus on the FPGA. Flush the pipeline cleanly on exception and restore correctly on `mret`.

Research question: What is the worst-case interrupt latency in cycles as a function of pipeline depth?

2.2 Category B — Memory Subsystem Optimisation (P8–P14)

P8: Cache associativity study

Integrate a configurable L1 data cache (direct-mapped, 2-way, 4-way; 256-byte capacity; parameterizable via a single `define`) with the memory-access stage. Drive the pipeline with memory-intensive programs and measure hit rate, stall cycles, and $F_{\max}$ for each configuration.

Research question: At what working-set size does 4-way associativity stop paying for its extra MUX logic?

P9: Write buffer for write-through cache

Add a 4-entry write buffer between the L1 write-through cache and a simulated main-memory model (20-cycle penalty). Compare no-buffer, 2-entry, and 4-entry configurations for stall-cycle absorption and synthesis area. Characterise the diminishing-returns depth empirically.

Research question: At what buffer depth do stall-reduction returns diminish, and is the area cost justified?

P10: Victim cache to reduce conflict misses

Add a 4-entry fully-associative victim cache between the L1 direct-mapped cache and main memory. Compare hit rate and CPI to a 4-way set-associative cache of the same total capacity on thrashing workloads.

Research question: For workloads that thrash a direct-mapped cache, how close does a direct-mapped + 4-entry victim cache get to 4-way in hit rate?

P11: DRAM address interleaving and bank parallelism

Implement a simplified DRAM controller mapping physical addresses across four banks using two strategies: row-column-bank and bank-row-column interleaving. Connect via a ready/valid handshake to the pipeline. Measure latency and throughput for sequential vs. strided access patterns.

Research question: Which address-mapping strategy yields lower average memory latency for stride-1 vs. stride-N access patterns?

P12: Software-managed scratchpad vs. hardware cache

Implement two data-side memory configurations: a conventional hardware-managed cache and an SRAM scratchpad accessible through a custom RISC-V instruction. Evaluate both with a matrix-multiply kernel. Quantify programmer effort vs. performance gain.

Research question: For a tightly-scheduled loop with predictable reuse, does explicit scratchpad management outperform hardware replacement policy?

P13: TLB design and page-fault handling

Implement a 4-entry fully-associative TLB for the data memory stage using Sv32 addressing. On a TLB miss, stall the pipeline and perform a 2-level page-table walk in a software trap handler. Report address-translation overhead as a function of program working-set size.

Research question: What is the effective address-translation overhead as a fraction of total execution time when TLB coverage is exceeded?

P14: Stride-based hardware prefetcher

Add a stride-based hardware prefetcher behind the L1 cache: track the last two miss addresses per memory instruction, detect a constant stride, and issue prefetch requests two strides ahead. Distinguish between helpful prefetches and harmful cache pollution across multiple access patterns.

Research question: What fraction of cold-start misses are eliminated, and under what stride patterns does the prefetcher cause harmful pollution?

2.3 Category C — ISA Extensions (P15–P21)**P15: RISC-V Zbb bit-manipulation unit**

Implement the RISC-V Zbb (basic bit manipulation) subset — `clz`, `ctz`, `cpop`, `rev8`, `sext.b`, `andn`, `orn`, `xnor` — as an extension to the ALU stage. Measure dynamic instruction-count reduction for a CRC-32 kernel and a popcount loop, and report the critical-path penalty added to the ALU.

Research question: What is the reduction in dynamic instruction count for bit-manipulation kernels and what is the ALU critical-path cost?

P16: Single-precision FPU co-processor interface

Connect a non-pipelined IEEE 754 single-precision FPU (reuse from Lab 1) to the RV32I pipeline via a custom-0 opcode. Handle multi-cycle latency with stall insertion. Analytically compare this loosely-coupled approach to a projected fully-pipelined FPU.

Research question: How does the CPI of loosely-coupled FPU integration compare to an ideally pipelined FPU for FP-heavy workloads?

P17: SIMD 8-bit packed arithmetic (P-extension subset)

Add four RISC-V packed SIMD instructions operating on 4×8 -bit lanes within a 32-bit register: `add8`, `sub8`, `max8`, `min8`. Implement in the execute stage without increasing the clock period. Evaluate on a histogram-computation and pixel-saturation kernel.

Research question: What is the speedup for image-processing kernels and what is the ALU area overhead compared to scalar code?

P18: Hardware loop counter instruction

Implement a RISC-V custom instruction `hwlp rd, rs1, imm` that sets a hardware loop count register and an end-of-loop PC register. On reaching the end-PC, the pipeline decrements the count and redirects without a branch instruction. Characterise hazard interaction with the forwarding unit.

Research question: What is the CPI improvement for tight fixed-count loops and how does `hwlp` interact with RAW hazards?

P19: MAC instruction and saturating arithmetic

Add `mac rd, rs1, rs2` (`rd += rs1 × rs2`, 32-bit) and `sadd/ssub` saturating add/subtract to the ALU. Handle the 2-cycle multiplier latency with stall and forwarding. Evaluate on an FIR filter kernel versus the pure RV32I implementation.

Research question: What is the instruction-count and CPI reduction for an FIR filter kernel using MAC vs. standard RV32I?

P20: RV32C compressed instruction decoder

Implement the RV32C 16-bit compressed instruction decoder as a pre-decode stage before fetch. The decoder expands 16-bit instructions to their 32-bit equivalents before the existing pipeline. Measure code-size reduction and the $F_{\max}$ penalty introduced by the pre-decode stage.

Research question: What is the code-size reduction for a typical program and what $F_{\max}$ penalty does the pre-decode stage introduce?

P21: Memory-mapped UART and interrupt-driven I/O

Implement a memory-mapped register interface for a simple UART transmitter (8N1, parameterizable baud rate). Connect it to the pipeline's data bus and write a RISC-V assembly driver. Compare polling vs. interrupt-driven I/O and characterise the impact on overall CPI.

Research question: How does polling vs. interrupt-driven I/O differ in CPU utilisation overhead and what is the effect on average CPI?

3 3 Group Projects (G1–G5) — 2 Students per Group

G1: Out-of-order execution with Tomasulo's algorithm (limited ROB)

Implement a 4-entry reorder buffer (ROB) with three reservation-station classes (ALU, Load, Store). In-order fetch and commit; out-of-order execute. Divide work between members: one handles the ROB and commit logic, the other handles reservation stations and the Common Data Bus (CDB) broadcast. Synthesize on Zynq and compare LUT count, $F_{\max}$, and CPI against the base core on a suite of instruction-level-parallelism benchmarks.

Research question: What is the actual IPC gain over an in-order base as a function of ILP in RV32I benchmark loops, and what is the area overhead?

G2: Dual-core RISC-V with shared L1 cache and MSI coherence

Instantiate two 5-stage RV32I cores sharing a common SRAM and implement a directory-less, snooping-based MSI protocol over a shared bus. Both cores must synthesize independently and run independent instruction streams. Measure bus contention, coherence miss overhead, and the conditions under which the shared cache becomes the bottleneck. Propose at least one architectural modification (e.g., per-core write buffers) and analytically predict its effect.

Research question: Under what workload mix does the shared cache saturate, and how does a per-core write buffer change the throughput-contention trade-off?

G3: Hardware branch predictor auto-tuning via performance counters

Implement a gshare predictor augmented with RISC-V M-mode performance counters (`minstret`, `mcycle`, plus a custom `mispred_count` CSR). Write a firmware loop that reads counters every N cycles and dynamically adjusts the predictor history length (2-bit to 8-bit) by switching between parameterized configurations through a configuration register. Frame the writeup as a hardware-firmware co-design / feedback control study.

Research question: Does closed-loop history-length adaptation improve prediction accuracy over a fixed-history predictor across workloads with differing branch behaviour?

G4: Two-level cache hierarchy with configurable replacement policies

Implement a 512-byte 4-way L1 data cache and a 2KB 2-way L2 cache with a run-time configurable replacement policy register (LRU, pseudo-LRU, FIFO, random). The L2 miss penalty to simulated DRAM is 20 cycles. Compare all four policies across at least three distinct access patterns (sequential, strided, random) for miss rate at both levels, total stall cycles, and FPGA resource consumption. Derive a simple analytical cost model from the results.

Research question: Can a simple analytical model predict which replacement policy minimises miss rate for a given working-set size ratio, and does hardware complexity justify LRU over pseudo-LRU?

G5: RISC-V with hardware LZ4-style decompression accelerator and DMA

Implement the RV32C compressed instruction decoder (cf. P20) plus a lightweight hardware decompression engine accessible via memory-mapped registers and a basic DMA handshake (source address, destination address, length, go/done registers). The accelerator shares the data bus with the pipeline. Measure end-to-end decompression throughput of the hardware accelerator vs. a software implementation on the same core, accounting for bus contention. Position the contribution relative to published ISA compression acceleration work.

***Research question:** What end-to-end throughput gain does a hardware decompression accelerator provide, and how does bus sharing with the pipeline core affect peak throughput?*

4 4 Deliverables & format for final report

All deliverables must be submitted as a single `.zip` archive on the LMS before the deadline (to be announced). Late submissions attract a **penalty**.

Deliverable	Individual	Group
Synthesizable RTL (Verilog/SystemVerilog)	✓	✓
Testbench to checking	✓	✓
Vivado synthesis + implementation report	✓	✓
Analysis table (baseline vs. modified)	✓	✓
Technical report in IEEE two-column format	✓	✓
Contribution statement	—	✓

Writeup structure (IEEE two-column format): *LaTeX template will be provided*

- Abstract (4–6 sentences)
- Introduction and motivation — state the research question explicitly
- Design and implementation — datapath changes, key design decisions
- Results — CPI table, synthesis numbers, graphs where applicable
- Conclusion — direct quantitative answer to the research question
- References (minimum 3 for projects report)